



*Advanced AI-based dynamic Integration
of heterogeneous data for semantic
clusters formalisation in human-centric
intelligent enterprise systems*

Exploring Advanced Ontology-Based Architectures for RDF Triple Extraction

Mario Lezoche

Thomas Pallet

Tuesday 10th March 2026

Table of content

01

Introduction

02

Dataset

03

Architecture 1

03

Architecture 4

04

Conclusion

Table of content

01

Introduction

02

Dataset

03

Architecture 1

03

Architecture 4

04

Conclusion

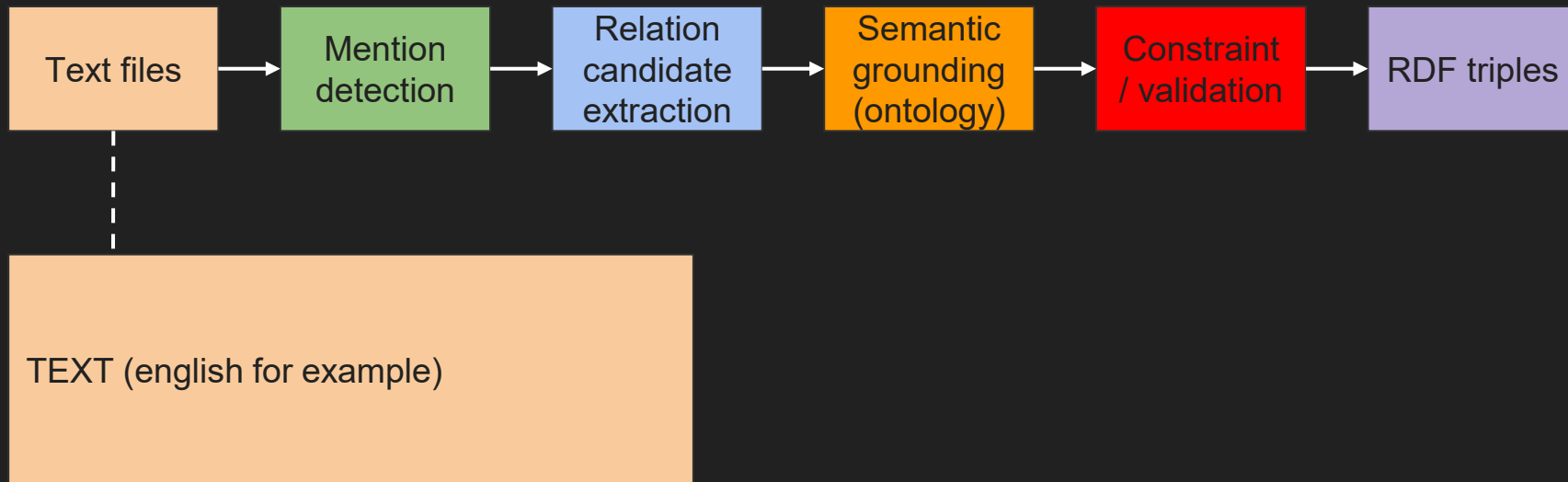
Introduction

Goal : develop and evaluate an ontology-based, supervised pipeline (NLP, ...) for automatic triple extraction from text.



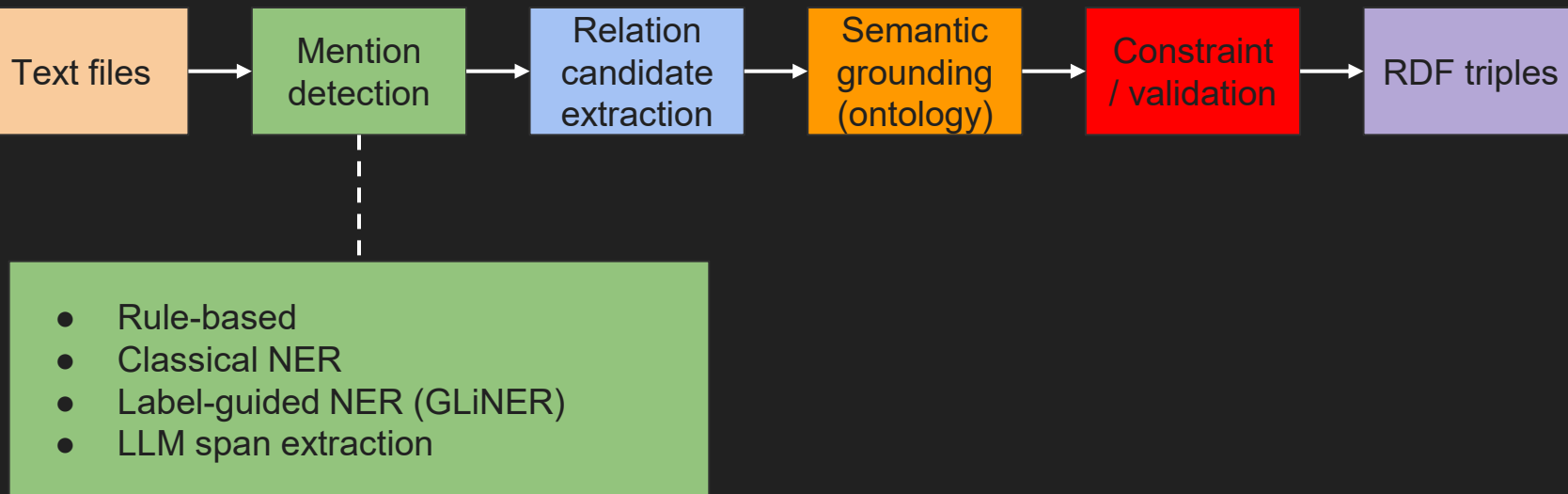
Introduction

Text files : plain text



Introduction

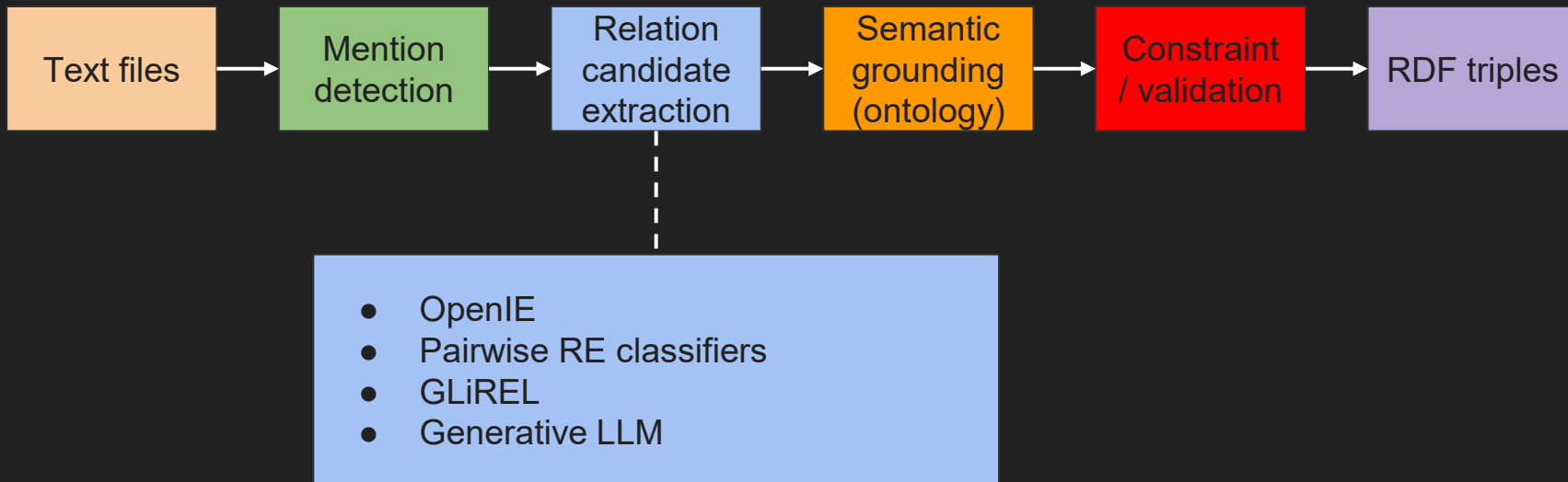
Mention detection : entities, events, attributes, literals (dates, numbers)



Introduction

Relation candidate extraction :

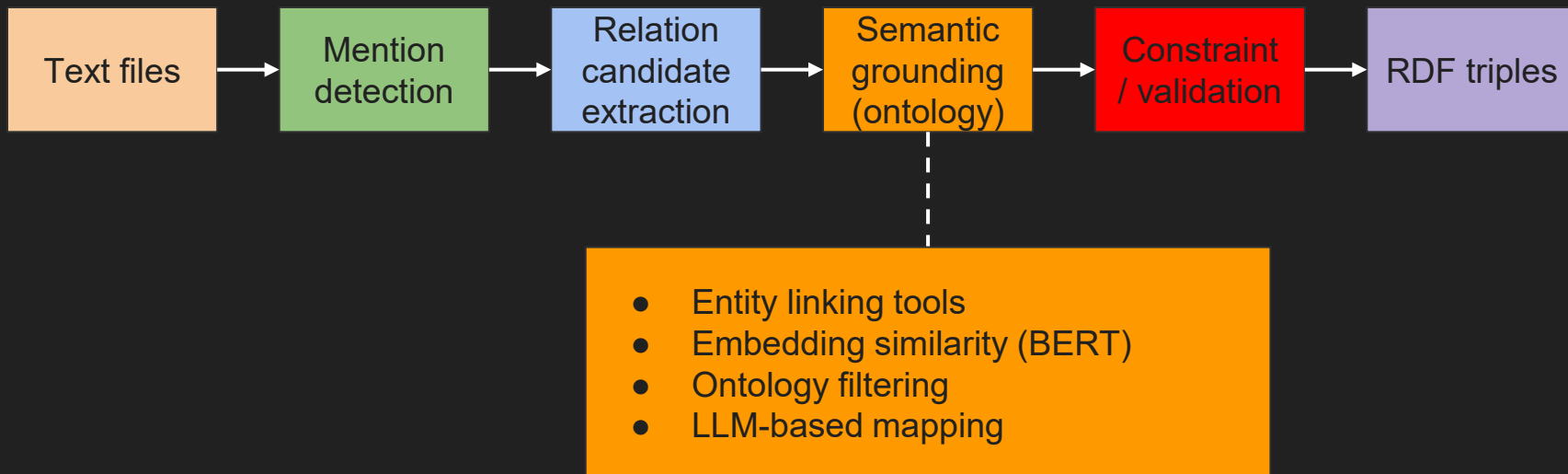
- A. Open (text) vs constrained (ontology-driven)
- B. Local (sentence-level) vs global (document-level)



Introduction

Semantic grounding :

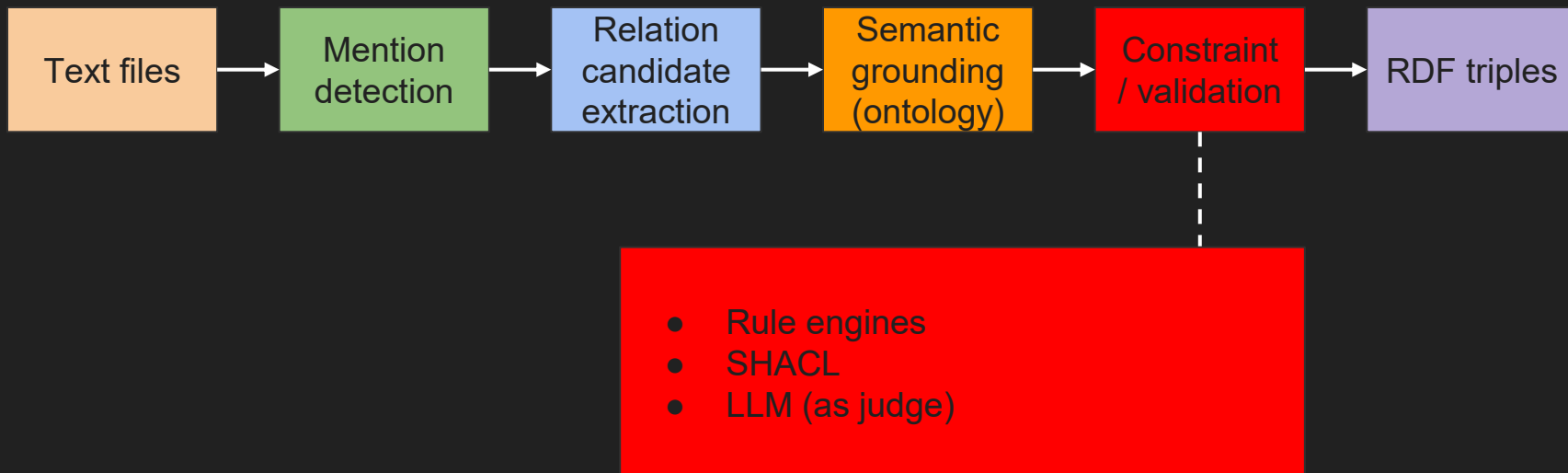
- Entity grounding (mentions $\rightarrow$ URIs)
- Relation grounding (relation $\rightarrow$ ontology property)
- Type grounding (entity $\rightarrow$ class)



Introduction

Constraint / validation :

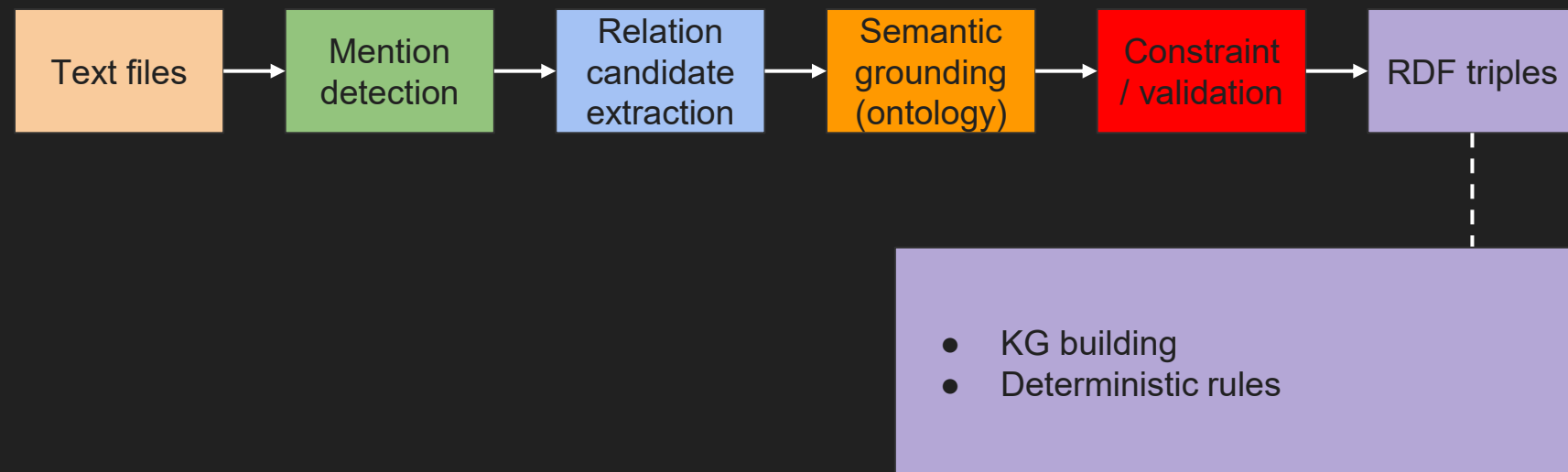
- Domain / range
- Cardinality



Introduction

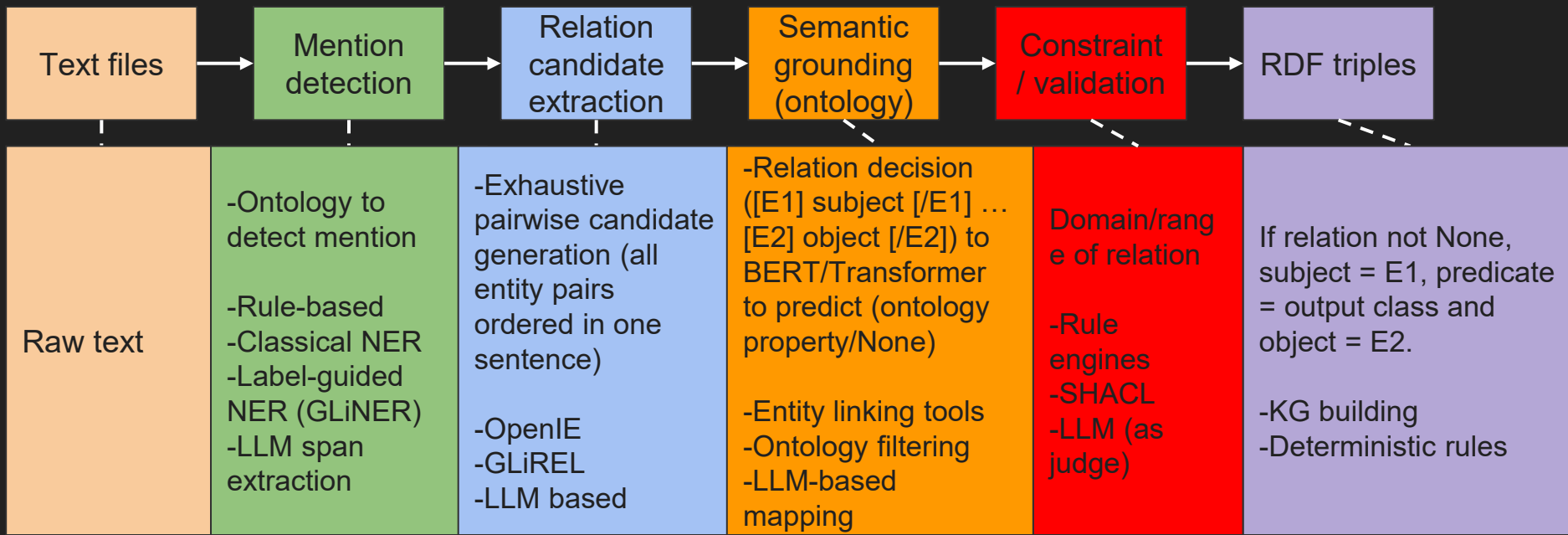
RDF triples formalisation :

- URI normalization, literals, datatypes
- Confidence score



Introduction : what I did

Goal : develop and evaluate an ontology-based, supervised pipeline (NLP, ...) for automatic triple extraction from text.



Introduction : what I did

Problems: Everything depend on the ontology, so if ontology changes, model is no longer well-defined

So it is like a compilation model, an ontology-compiled supervision model because one compiled, it doesn't take into account the changes into my ontology !

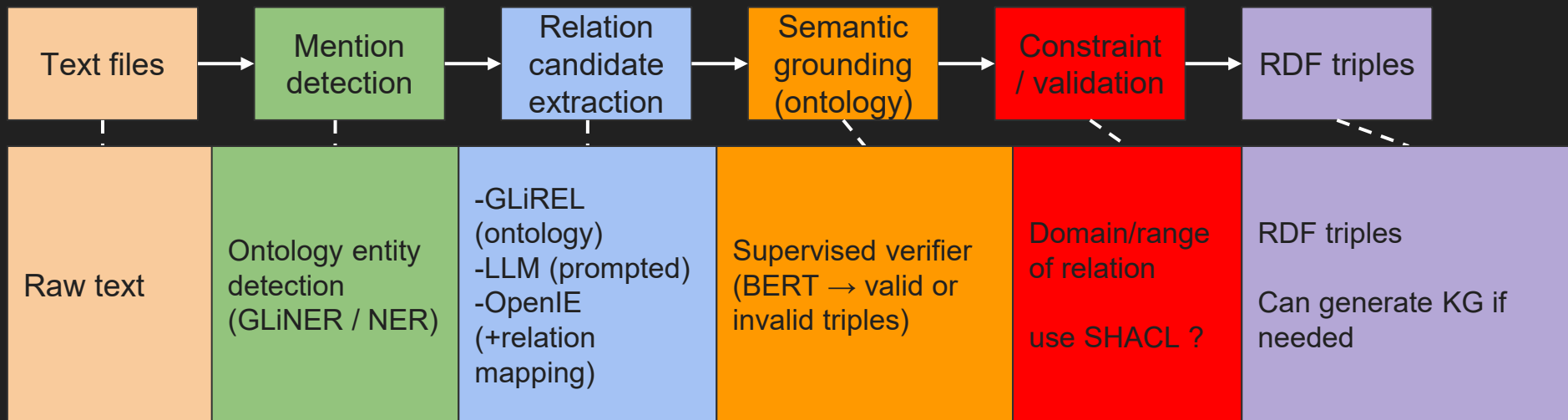
GLiNER: good for dynamic ontologies at the entity levels (add class without retraining, rename class) but can't decide/classify relations

GLiREL: schema relation extractor (relation as text labels & external to the model itself)

LLM: very good here but depends prompt and are very costly (computation time, hallucination, latency, non-determinism, ...)

Architecture 1: Schema Extraction & Supervised verifier

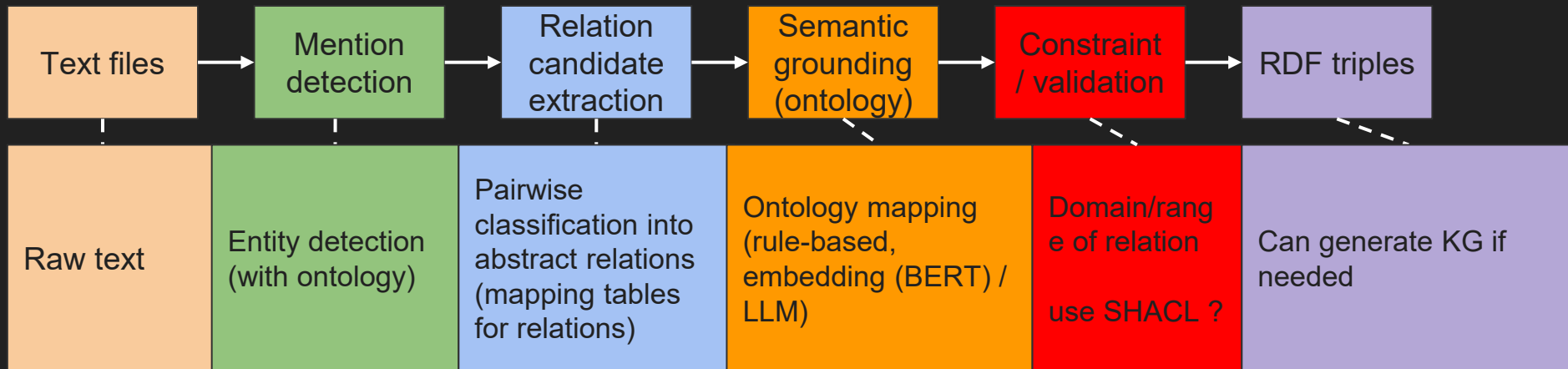
Goal is to dissociate extraction (dynamic) from the final decision (supervised, stable)



- Ontology changes → update the label list
- No dataset rebuild
- BERT is no more the **generator** **BUT** the **verifier** ! It doesn't predict relation anymore but verify them !

Architecture 2: abstract relation & ontology mapping

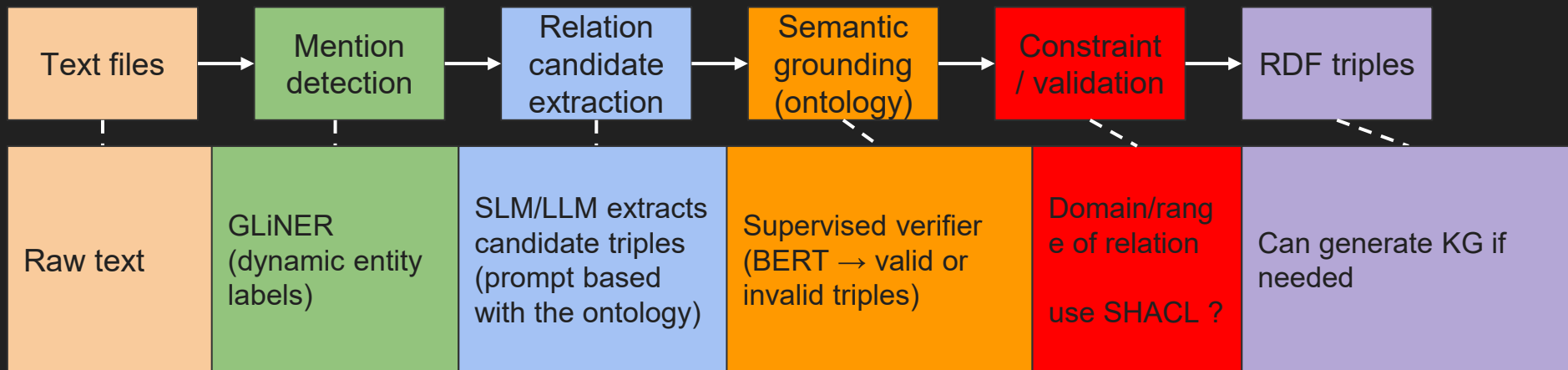
Stabilize learning by predicting abstract relationships, independent of concrete ontology (same goal as architecture 1 = dynamic)



- Add ontology predication == update the mapping table ?
- Similar to what I did but try to be dynamic

Architecture 3: ontology with LLM & Supervised verifier

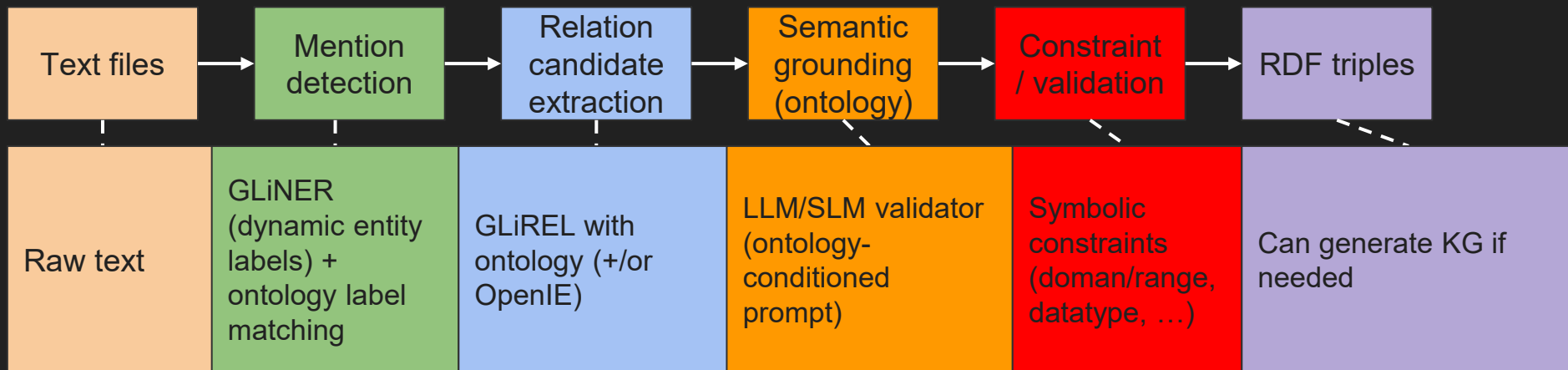
Maximize flexibility and semantic coverage with LLMs, while maintaining a supervised security nest.



- Very flexible and ontology based (so should prevent the hallucination problem)
- Cons: depend on the quality of the prompt and LLM are costly (computation, latency, ...) so maybe SLM ?

Architecture 4: dynamic candidate extraction + LLM validator & symbolic constraints

GLiNER/labels detect mentions, GLiREL proposes candidates, LLM validates semantics, SHACL enforces ontology truth, RDF stores final triples.



- **Pro:** Highly dynamic with ontology evolution (new entities/relations can be handled at inference time, no mandatory immediate retraining).
- **Con:** Higher complexity/cost and non-deterministic validator behavior (prompt quality + LLM variability require strict constraints and monitoring).

Table of content

01

Introduction

02

Dataset

03

Architecture 1

03

Architecture 4

04

Conclusion

Dataset creation

I have to rebuild the dataset again because now in all my architectures, the BERT-like model is no more the generator of the triples but is the validator !

But, I used my previous experience to not repeat previous mistakes I made for the generation of the dataset

- 4 tier for the dataset, 250 txt each and json (annotation) associated, 4 increasing levels in term of increasing difficulty
- Splitting now the dataset in 70% training, 15% validation and 15% testing (no sentence leak between group)
- Reminder that during the generation of those files, it is first only txt and associated json annotation

Dataset creation: example

txt: “Based on the scouting report, Flying is one of the types assigned to Charizard.”

annotation:

```
{  
  "text": "Based on the scouting report, Flying is one of the types assigned  
to Charizard.",  
  "triples": [  
    {  
      "subject": "http://example.org/pokemon#species_0006",  
      "predicate": "http://example.org/pokemon#hasType",  
      "object": "http://example.org/pokemon#Flying"  
    }  
  ]  
},
```

Dataset: stats

So, now I have my dataset, 1000 txt files, here is a recap of the composition :

- 40 246 sentences, 7 750 sentences with a triples to predict and 32 496 without triples
→ 80% with negative prediction and 20% with positive prediction

	NEG (label:None)	POS (valid triples)	canLearnMove	hasMoveType	evolvesFrom	hasType	evolvesTo
Global	32 496	7 750	3 027	1074	854	1976	816
Training	22 448	5 412	2089	758	589	1395	581
Validation	5 093	1 171	464	305	123	122	157
Testing	4 955	1 167	474	279	143	112	159

Dataset creation

Now, with this consideration, I can build the dataset that I will really use to train a model inside architecture:

```
"Bulbasaur has the Grass type."
```

```
{"text": "[REL] hasType [/REL] [E1]Bulbasaur[/E1] has the [E2]Grass[/E2] type.", "label": "VALID",  
"candidate_relation": "http://example.org/pokemon#hasType", "subject":  
"http://example.org/pokemon#species_0001", "object": "http://example.org/pokemon#Grass", "tier": "tier_1",  
"sentence": "Bulbasaur has the Grass type."}
```

```
{"text": "[REL] evolvesTo [/REL] [E1]Bulbasaur[/E1] has the [E2]Grass[/E2] type.", "label": "INVALID",  
"candidate_relation": "http://example.org/pokemon#evolvesTo", "subject":  
"http://example.org/pokemon#species_0001", "object": "http://example.org/pokemon#Grass", "tier": "tier_1",  
"sentence": "Bulbasaur has the Grass type."}
```

→ model only need text and the label, the rest is metadata (for human interpretation)

Dataset creation

It is important to give to the model what my pipeline from raw txt will give too, to perform better, giving the relation tested and the two entities tested is important

```
"[REL] hasType [/REL] [E1]Bulbasaur[/E1] has the [E2]Grass[/E2] type."
```

Now, my model will predict only whether or no the relation is valid (it is not only work of domain and range because my model has to make prediction on its own, then only if the prediction might be valid but the domain and range are not, then I can still control the validation of such triples)

→ Built 3 jsonl, now dataset is ready !

Table of content

01

Introduction

02

Dataset

03

Architecture 1

04

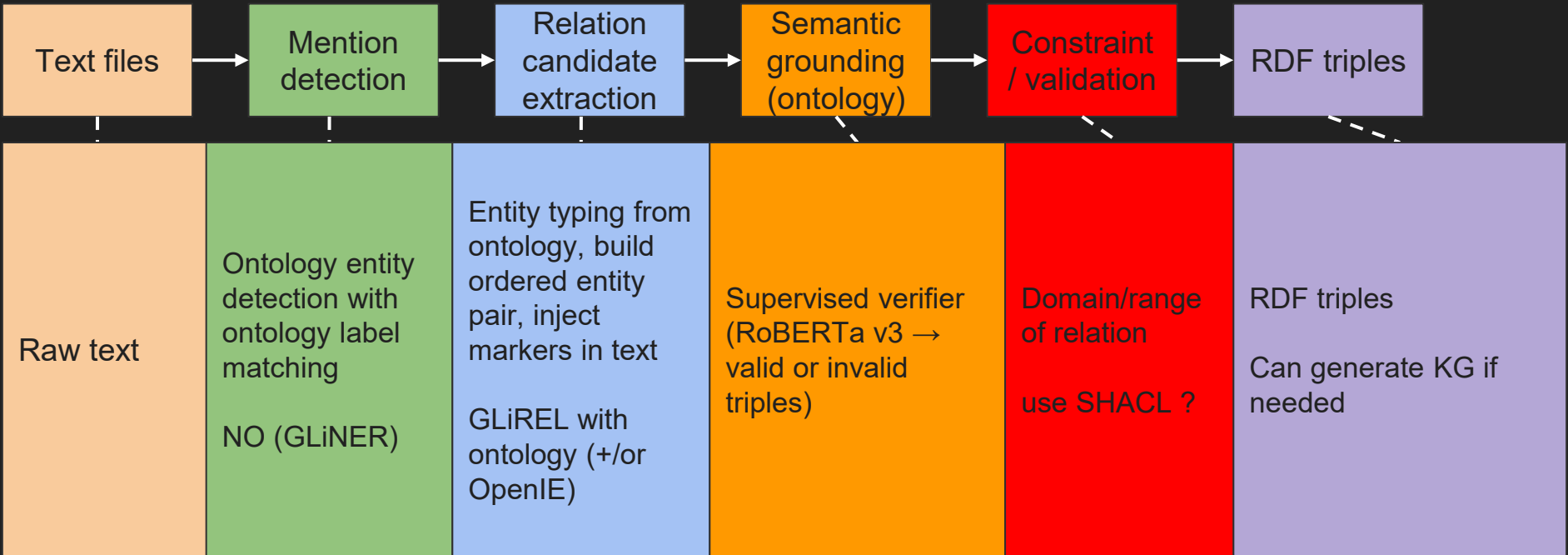
Architecture 4

05

Conclusion

Architecture 1: Schema Extraction & Supervised verifier

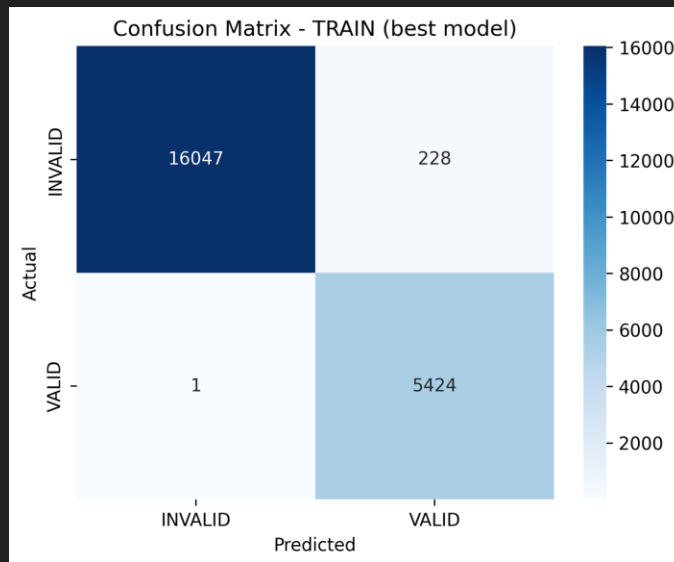
Goal is to dissociate extraction (dynamic) from the final decision (supervised, stable)



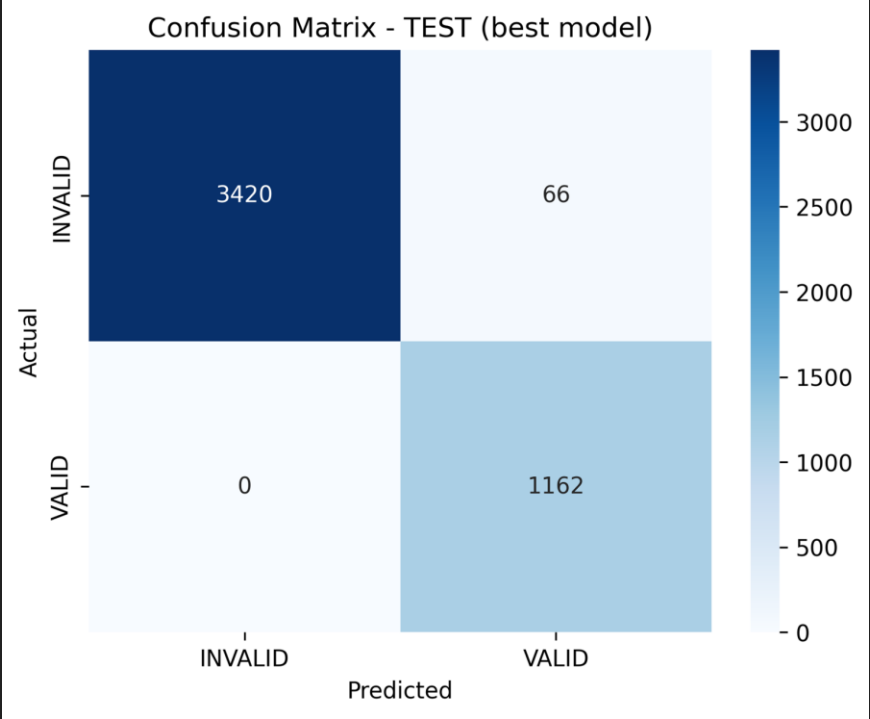
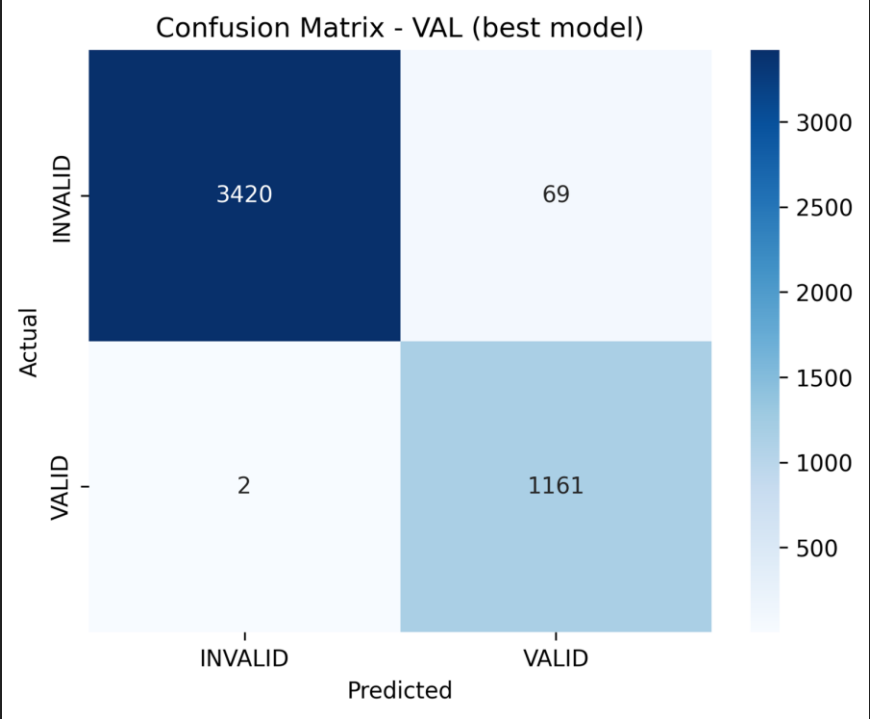
Architecture 1: training

To train, use colab

Use DL technics, scheduler, warmup, batch-size, epochs, learning rate, keep best model (better f1 score), ...



Architecture 1: validation & testing



Architecture 1: testing v2

Same process as before, I generated 10 texts and then evaluate them on my whole pipeline. Those text are not generated with the same LLM so they have to be different in terms of construction of the sentences

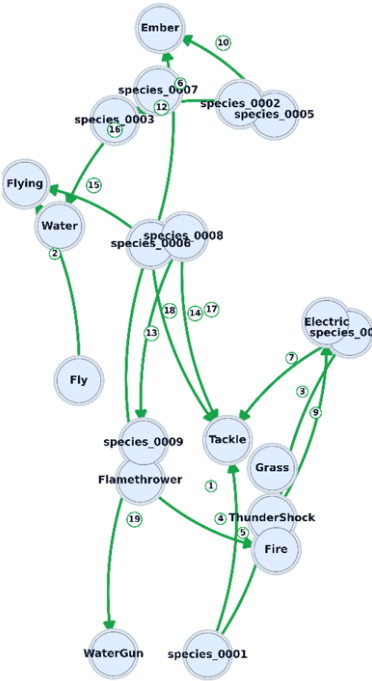
Results RoBERTa v3 model

Story	TP	FP	FN	Precision (%)	Recall (%)	F1 (%)
story_01	17	0	3	100 %	85.0 %	91.9 %
story_02	17	0	3	100 %	85.0 %	91.9 %
story_03	17	0	3	100 %	85.0 %	91.9 %
story_04	18	0	2	100 %	90.0 %	94.7 %
story_05	19	0	1	100 %	95.0 %	97.4 %
story_06	18	0	2	100 %	90.0 %	94.7 %
story_07	18	0	2	100 %	90.0 %	94.7 %
story_08	18	0	2	100 %	90.0 %	94.7 %
story_09	17	0	3	100 %	85.0 %	91.9 %
story_10	17	0	3	100 %	85.0 %	91.9 %
TOTAL (mean)	-	-	-	100 %	88.0 %	93.6 %

Story 3

Relation Graph Comparison - story_03 | Model=UNKNOWN | TP=17 FP=0 FN=3 | P=1.000 R=0.850 F1=0.919

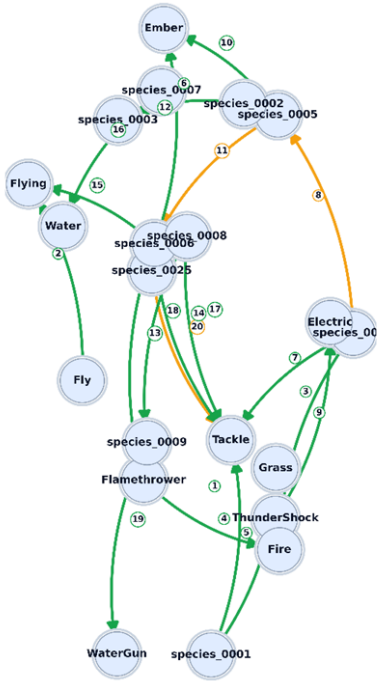
Predicted Graph - story_03 (green=TP, red=FP)



Edge list

- 01. Flamethrower -> Fire : hasMoveType [TP]
- 02. Fly -> Flying : hasMoveType [TP]
- 03. ThunderShock -> Electric : hasMoveType [TP]
- 04. species_0001 -> Tackle : canLearnMove [TP]
- 05. species_0001 -> Grass : hasType [TP]
- 06. species_0002 -> species_0003 : evolvesTo [TP]
- 07. species_0004 -> Tackle : canLearnMove [TP]
- 09. species_0004 -> Fire : hasType [TP]
- 10. species_0005 -> Ember : canLearnMove [TP]
- 12. species_0005 -> Ember : canLearnMove [TP]
- 13. species_0006 -> Flamethrower : canLearnMove [TP]
- 14. species_0006 -> Tackle : canLearnMove [TP]
- 15. species_0006 -> Flying : hasType [TP]
- 16. species_0007 -> Water : hasType [TP]
- 17. species_0008 -> Tackle : canLearnMove [TP]
- 18. species_0008 -> species_0009 : evolvesTo [TP]
- 19. species_0009 -> WaterGun : canLearnMove [TP]

Actual Graph - story_03 (green=TP, orange=FN)



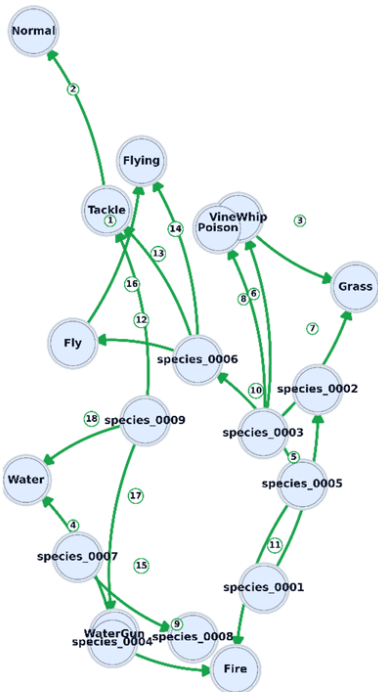
Edge list

- 01. Flamethrower -> Fire : hasMoveType [TP]
- 02. Fly -> Flying : hasMoveType [TP]
- 03. ThunderShock -> Electric : hasMoveType [TP]
- 04. species_0001 -> Tackle : canLearnMove [TP]
- 05. species_0001 -> Grass : hasType [TP]
- 06. species_0002 -> species_0003 : evolvesTo [TP]
- 07. species_0004 -> Tackle : canLearnMove [TP]
- 08. species_0004 -> species_0005 : evolvesTo [FN]
- 09. species_0004 -> Fire : hasType [TP]
- 10. species_0005 -> Ember : canLearnMove [TP]
- 11. species_0005 -> species_0006 : evolvesTo [FN]
- 12. species_0006 -> Ember : canLearnMove [TP]
- 13. species_0006 -> Flamethrower : canLearnMove [TP]
- 14. species_0006 -> Tackle : canLearnMove [TP]
- 15. species_0006 -> Flying : hasType [TP]
- 16. species_0007 -> Water : hasType [TP]
- 17. species_0008 -> Tackle : canLearnMove [TP]
- 18. species_0008 -> species_0009 : evolvesTo [TP]
- 19. species_0009 -> WaterGun : canLearnMove [TP]
- 20. species_0025 -> Tackle : canLearnMove [FN]

Story 8

Relation Graph Comparison - story_08 | Model=UNKNOWN | TP=18 FP=0 FN=2 | P=1.000 R=0.900 F1=0.947

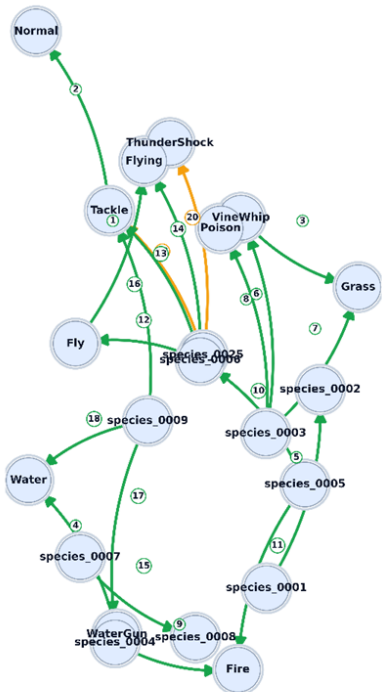
Predicted Graph - story_08 (green=TP, red=FP)



Edge list

- 01. Fly -> Flying : hasMoveType [TP]
- 02. Tackle -> Normal : hasMoveType [TP]
- 03. VineWhip -> Grass : hasMoveType [TP]
- 04. WaterGun -> Water : hasMoveType [TP]
- 05. species_0001 -> species_0002 : evolvesTo [TP]
- 06. species_0003 -> VineWhip : canLearnMove [TP]
- 07. species_0003 -> Grass : hasType [TP]
- 08. species_0003 -> Poison : hasType [TP]
- 09. species_0004 -> Fire : hasType [TP]
- 10. species_0005 -> species_0006 : evolvesTo [TP]
- 11. species_0005 -> Fire : hasType [TP]
- 12. species_0006 -> Fly : canLearnMove [TP]
- 13. species_0006 -> Tackle : canLearnMove [TP]
- 14. species_0006 -> Flying : hasType [TP]
- 15. species_0007 -> species_0008 : evolvesTo [TP]
- 16. species_0009 -> Tackle : canLearnMove [TP]
- 17. species_0009 -> WaterGun : canLearnMove [TP]
- 18. species_0009 -> Water : hasType [TP]

Actual Graph - story_08 (green=TP, orange=FN)



Edge list

- 01. Fly -> Flying : hasMoveType [TP]
- 02. Tackle -> Normal : hasMoveType [TP]
- 03. VineWhip -> Grass : hasMoveType [TP]
- 04. WaterGun -> Water : hasMoveType [TP]
- 05. species_0001 -> species_0002 : evolvesTo [TP]
- 06. species_0003 -> VineWhip : canLearnMove [TP]
- 07. species_0003 -> Grass : hasType [TP]
- 08. species_0003 -> Poison : hasType [TP]
- 09. species_0004 -> Fire : hasType [TP]
- 10. species_0005 -> species_0006 : evolvesTo [TP]
- 11. species_0005 -> Fire : hasType [TP]
- 12. species_0006 -> Fly : canLearnMove [TP]
- 13. species_0006 -> Tackle : canLearnMove [TP]
- 14. species_0006 -> Flying : hasType [TP]
- 15. species_0007 -> species_0008 : evolvesTo [TP]
- 16. species_0009 -> Tackle : canLearnMove [TP]
- 17. species_0009 -> WaterGun : canLearnMove [TP]
- 18. species_0009 -> Water : hasType [TP]
- 19. species_0025 -> Tackle : canLearnMove [FN]
- 26. species_0025 -> ThunderShock : canLearnMove [FN]
- ...

Architecture 1: testing v2

Results are very good but I realize something when testing this

In a way it is dynamic but I will still need to retrain my model so it updated and validate the valid relations

But it showed me that putting the BERT-like model as a validator was the good approach, but I think that a BERT model can't be suited for a dynamic approach

dynamic = evolution of the ontology (classes, properties, ...)

Architecture 1: limitations

So, I won't be using the architecture 2&3 because I think they got the same issue.

I wanna try another one, the architecture 4, that I think is suited for the dynamic part but maybe has other issues !

Table of content

01

Introduction

02

Dataset

03

Architecture 1

04

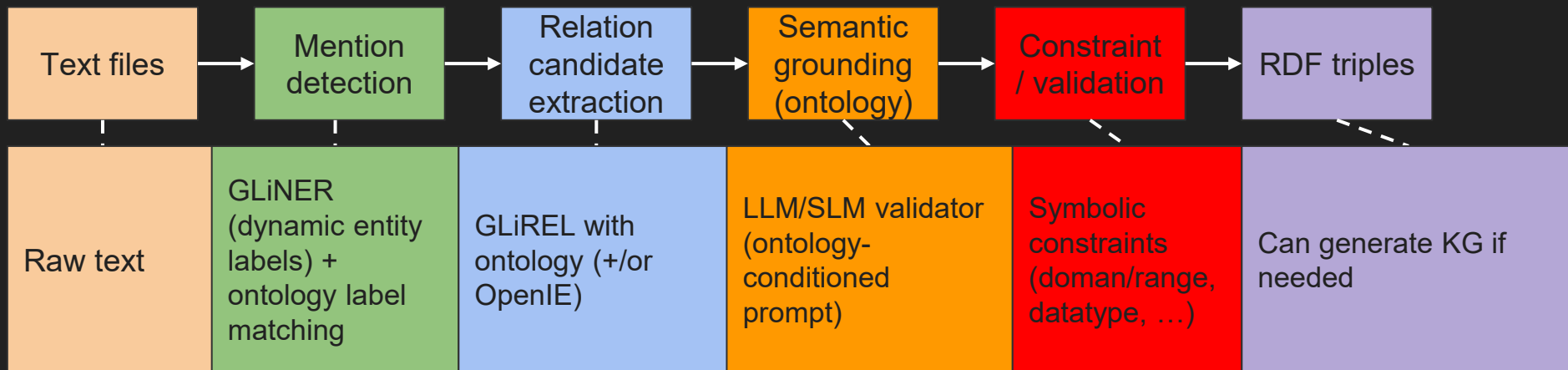
Architecture 4

05

Conclusion

Architecture 4: dynamic candidate extraction + LLM validator & symbolic constraints

GLiNER/labels detect mentions, GLiREL proposes candidates, LLM validates semantics, SHACL enforces ontology truth, RDF stores final triples.



- **Pro:** Highly dynamic with ontology evolution (new entities/relations can be handled at inference time, no mandatory immediate retraining).
- **Con:** Higher complexity/cost and non-deterministic validator behavior (prompt quality + LLM variability require strict constraints and monitoring).

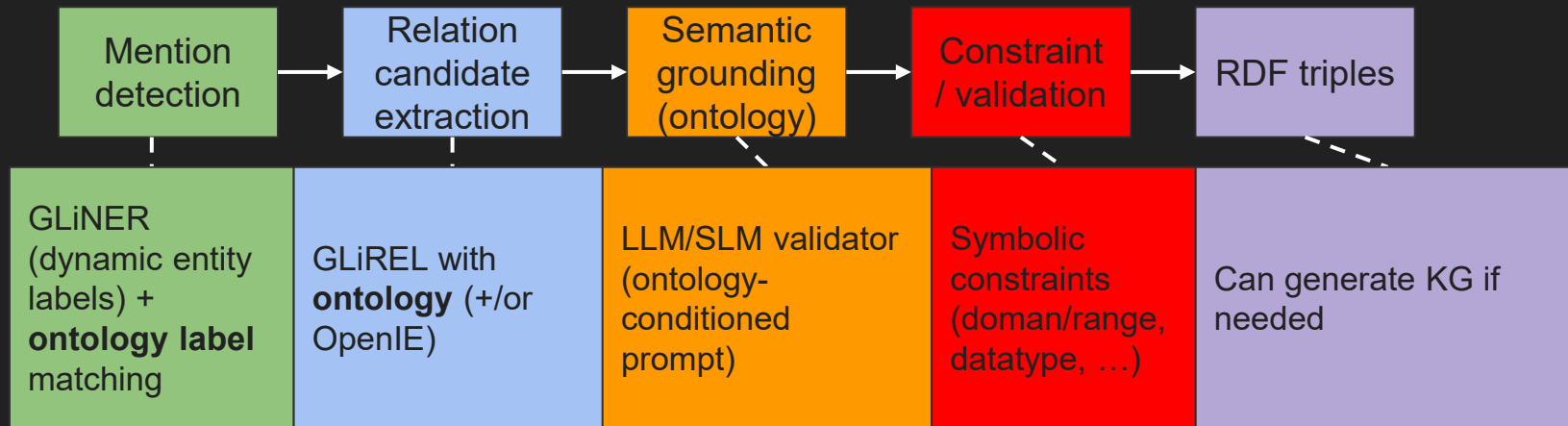
Architecture 4: presentation

Obviously, the really big problem is that putting the validation from my LLM is very dangerous if we want an explainable AI, but given that we have an ontology, we can rely on it to prevent hallucination problem and to help us explainable the validation or not, give a confidence score for example.

When using LLM, you can do everything you want with the ontology, everything because the GLINER and GLIREL will always be ontology-based so okay for the entity detection and the entity relation !

And then, doing a custom prompt to the LLM/SLM will make that the pipeline is actually dynamic, but I think important in the prompt to provide example.

Architecture 4: dynamic candidate extraction + LLM validator & symbolic constraints



Trade-off, it is obviously the computationally expensive and prompt-sensitive LLM/SLM

But, have to try because with enough gating or catching, it can be possible to control it and then gain this dynamic part, which is the key to everything then (choosing LLM/SLM adapted and so on, here it is just a benchmarking, just a test here)

Architecture 4: consideration

Used ollama

pulled qwen2.5b to try

30 seconds to have an answer of a validation for a triple, that's the computational cost right there

Maybe a 2-step model

1st model = SLM

2nd model = LLM, called if confidence score is not high (but how is it computed and is it reliable ?)

Table of content

01

Introduction

02

Dataset

03

Architecture 1

04

Architecture 4

05

Conclusion

Thank you for your attention !

Questions ?